

Universidad Tecnológica Metropolitana

Tecnologías de la Información Innovación Digital

Bases de datos en la nube

PROYECTO INTEGRADOR DE UNIDAD I

CLAUDENT: Sistema seguro de gestión de
información odontológica en la nube

Maestro: I.S. Lizandro Israel Reyes Carrillo

Equipo 2 — Integrantes:

Canul Cua Jesús Guadalupe

Peña Herrera Bogarth Alejandro

Rodríguez Rangel Yael

Solis Baas Diego Alberto

7° Cuatrimestre — Grupo A

Fecha de entrega: viernes 26 de septiembre de 2025

Índice General

1. Definición de problemática y requerimientos de información
2. Diseño de la arquitectura de datos
3. Documentación del proceso de normalización (1FN, 2FN, 3FN)
4. Índices y optimización de rendimiento
5. Consultas SQL analíticas de negocio (8 consultas)
6. Seguridad, roles y privilegios DCL
7. Pruebas de seguridad documentadas

1. Definición de problemática y requerimientos

Título del proyecto

CLAUDENT: Plataforma segura para la administración de pacientes, tratamientos y procesos administrativos de una clínica odontológica mediante una base de datos relacional en la nube.

Descripción del contexto real

Las clínicas odontológicas manejan información sensible relacionada con pacientes, tratamientos, citas, pagos y personal médico. Actualmente muchas clínicas pequeñas utilizan procesos manuales como expedientes físicos, hojas de cálculo y agendas tradicionales, provocando problemas de organización, pérdida de información y falta de control sobre los accesos.

La problemática aumenta cuando diferentes colaboradores requieren consultar información, ya que compartir cuentas administrativas o almacenar información sin controles adecuados puede provocar modificaciones accidentales, accesos no autorizados y pérdida de datos.

Problemática seleccionada

Falta de una plataforma centralizada y segura para administrar la información clínica y administrativa de una odontología, generando problemas de organización, seguridad y control de acceso a los datos.

Justificación técnica

El desarrollo de una base de datos relacional en la nube permitirá centralizar la información de pacientes, odontólogos, citas, tratamientos y pagos, además de implementar mecanismos de seguridad mediante roles, privilegios y auditoría. Al utilizar Supabase como proveedor DBaaS, se obtiene escalabilidad, alta disponibilidad y una interfaz moderna para la gestión de datos.

Objetivo general

Diseñar e implementar una base de datos relacional en la nube para una clínica odontológica que permita administrar información clínica y administrativa aplicando mecanismos de seguridad y control de acceso.

Objetivos específicos

- Técnico: Diseñar una arquitectura relacional normalizada en 3FN en PostgreSQL aplicando restricciones de integridad, índices de rendimiento y triggers de auditoría.
- Funcional: Gestionar información de pacientes, citas, tratamientos y pagos mediante consultas SQL analíticas complejas.
- De seguridad: Implementar 3 roles con privilegios granulares diferenciados (DCL) y documentar pruebas de acceso permitido y denegado.

Alcance

- Registro y administración de pacientes.
- Control de odontólogos y especialidades.
- Gestión de citas y consultas.
- Registro de tratamientos y pagos.

- Administración de usuarios y permisos.
- Auditoría automática de cambios mediante trigger.

Requerimientos funcionales

- Registrar pacientes con información personal única (teléfono, correo).
- Gestionar citas asociadas a pacientes y odontólogos con estados controlados.
- Registrar consultas/tratamientos realizados con diagnóstico.
- Controlar pagos relacionados con consultas por múltiples métodos.
- Registrar cambios automáticamente mediante bitácora de auditoría con trigger.

2. Diseño de la arquitectura de datos

La base de datos está diseñada bajo el modelo relacional y normalizada hasta tercera forma normal (3FN). El esquema consta de 10 tablas relacionadas mediante llaves primarias y foráneas, desplegadas dentro del esquema claudent en PostgreSQL (Supabase).

Tablas del modelo (10 tablas)

Tabla	Descripción
rol	Catálogo de perfiles de acceso del sistema.
usuario	Usuarios con acceso al sistema, vinculados a un rol.
especialidad	Catálogo de especialidades odontológicas.
dentista	Profesionales odontológicos con su especialidad.
paciente	Datos personales y de contacto de los pacientes.
servicio	Catálogo de tratamientos/servicios con costo.
cita	Agenda de citas entre pacientes y dentistas.
consulta	Registro de atención y servicio aplicado por cita.
pago	Pagos asociados a consultas realizadas.
bitacora_auditoria	Registro automático de cambios para auditoría.

Relaciones principales

- rol 1:N usuario
- especialidad 1:N dentista
- paciente 1:N cita
- dentista 1:N cita
- cita 1:1 consulta
- servicio 1:N consulta
- consulta 1:N pago
- usuario 1:N bitacora_auditoria

3. Documentación del proceso de normalización

Primera Forma Normal (1FN)

- Todos los campos contienen valores atómicos (indivisibles).
- No existen grupos repetitivos ni arrays en ninguna columna.
- Los nombres están separados en campos nombre y apellido en las tablas dentista y paciente.
- Las direcciones se almacenan como VARCHAR individual dado el alcance del proyecto.

Segunda Forma Normal (2FN)

- Se cumple 1FN y cada atributo no clave depende completamente de la clave primaria.
- Todas las claves primarias son simples (SERIAL), no existen claves compuestas.
- La tabla consulta relaciona cita y servicio mediante dos FKs independientes.

Tercera Forma Normal (3FN)

- Se cumple 2FN y no existen dependencias transitivas.
- Catálogos independientes: rol, especialidad y servicio están en tablas separadas referenciadas por FK.
- nombre_especialidad no depende transitivamente de id_dentista porque está en su propia tabla.
- La tabla consulta separa la relación cita-servicio evitando almacenar el servicio en la cita.
- Los estados usan CHECK ya que son dominios fijos de cada entidad.

Resultado: Las 10 tablas cumplen 1FN, 2FN y 3FN. Catálogos separados como tablas independientes con FK eliminan redundancias y dependencias transitivas.

4. Índices y optimización de rendimiento

Se crearon 14 índices estratégicos para acelerar las búsquedas más frecuentes del sistema.

Índice	Columna(s)	Justificación
idx_cita_fecha	cita(fecha_cita)	Agenda diaria/semanal
idx_cita_estado	cita(estado)	Filtrar por estado
idx_cita_paciente	cita(id_paciente)	Historial del paciente
idx_cita_dentista	cita(id_dentista)	Agenda del dentista
idx_paciente_apellido	paciente(apellido)	Búsqueda en recepción
idx_pac_nom_ape	paciente(nombre, apellido)	Búsqueda combinada
idx_consulta_fecha	consulta(fecha_consulta)	Reportes diarios
idx_pago_fecha	pago(fecha_pago)	Cortes de caja
idx_pago_metodo	pago(metodo_pago)	Reportes financieros
idx_bitacora_fecha	bitacora(fecha_evento)	Auditoría temporal
idx_bitacora_tabla	bitacora(tabla_afectada)	Auditoría por entidad
idx_dent_espec	dentista(id_especialidad)	Filtrar por especialidad
idx_usuario_rol	usuario(id_rol)	Filtrar por tipo de rol

Trigger de auditoría automática

Se implementó un trigger que registra automáticamente INSERT, UPDATE y DELETE sobre la tabla paciente en bitacora_auditoria.

```
CREATE OR REPLACE FUNCTION fn_auditoria_paciente()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO claudent.bitacora_auditoria
        (operacion, tabla_afectada, descripcion)
    VALUES (TG_OP, 'paciente',
        CASE
            WHEN TG_OP = 'INSERT' THEN 'Nuevo: ' || NEW.nombre
            WHEN TG_OP = 'UPDATE' THEN 'Actualizado ID: ' || OLD.id_paciente
            WHEN TG_OP = 'DELETE' THEN 'Eliminado: ' || OLD.nombre
        END);
    RETURN COALESCE(NEW, OLD);
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditoria_paciente
AFTER INSERT OR UPDATE OR DELETE ON paciente
FOR EACH ROW EXECUTE FUNCTION fn_auditoria_paciente();
```

Impacto en rendimiento

- Búsquedas por fecha: sequential scan -> index scan, $O(n)$ -> $O(\log n)$.
- Índice compuesto paciente(nombre, apellido) permite búsquedas parciales y exactas.
- Índices en pago permiten reportes financieros sin full table scan.

5. Consultas SQL analíticas de negocio

Se diseñaron 8 consultas SQL complejas con INNER JOIN, LEFT JOIN, COUNT, SUM, AVG, GROUP BY, HAVING y ORDER BY.

Consulta 1: Pacientes con citas programadas

INNER JOIN entre paciente, cita y dentista.

```
SELECT p.nombre || ' ' || p.apellido AS paciente,
       d.nombre || ' ' || d.apellido AS dentista,
       c.fecha_cita, c.hora_cita, c.motivo
FROM cita c
INNER JOIN paciente p ON c.id_paciente = p.id_paciente
INNER JOIN dentista d ON c.id_dentista = d.id_dentista
WHERE c.estado = 'PROGRAMADA'
ORDER BY c.fecha_cita, c.hora_cita;
```

Consulta 2: Pacientes atendidos por odontólogo

COUNT DISTINCT + GROUP BY + LEFT JOIN.

```
SELECT d.nombre || ' ' || d.apellido AS dentista,
       e.nombre_especialidad,
       COUNT(DISTINCT c.id_paciente) AS pacientes
FROM dentista d
INNER JOIN especialidad e
      ON d.id_especialidad = e.id_especialidad
LEFT JOIN cita c ON d.id_dentista = c.id_dentista
      AND c.estado = 'ATENDIDA'
GROUP BY d.id_dentista, d.nombre, d.apellido,
         e.nombre_especialidad
ORDER BY pacientes DESC;
```

Consulta 3: Ingresos totales por servicio

SUM + AVG + COUNT cruzando servicio-consulta-pago.

```
SELECT s.nombre_servicio,
       COUNT(co.id_consulta) AS veces_realizado,
       SUM(pa.monto) AS ingreso_total,
       AVG(pa.monto) AS ingreso_promedio
FROM servicio s
INNER JOIN consulta co ON s.id_servicio = co.id_servicio
INNER JOIN pago pa ON co.id_consulta = pa.id_consulta
GROUP BY s.id_servicio, s.nombre_servicio
ORDER BY ingreso_total DESC;
```

Consulta 4: Análisis por método de pago

AVG + SUM + COUNT + MIN + MAX + GROUP BY.

```
SELECT metodo_pago,
       COUNT(*) AS cantidad_pagos,
       SUM(monto) AS total_recaudado,
       AVG(monto) AS promedio_pago,
       MIN(monto) AS pago_minimo,
       MAX(monto) AS pago_maximo
FROM pago
GROUP BY metodo_pago
ORDER BY total_recaudado DESC;
```

Consulta 5: Tratamientos más solicitados (HAVING)

COUNT + GROUP BY + HAVING filtra servicios populares.

```
SELECT s.nombre_servicio, s.costo,
       COUNT(co.id_consulta) AS veces_solicitado,
       SUM(pa.monto) AS ingresos_generados
FROM servicio s
INNER JOIN consulta co ON s.id_servicio = co.id_servicio
INNER JOIN pago pa ON co.id_consulta = pa.id_consulta
GROUP BY s.id_servicio, s.nombre_servicio, s.costo
HAVING COUNT(co.id_consulta) >= 1
ORDER BY veces_solicitado DESC;
```

Consulta 6: Pacientes sin citas (LEFT JOIN + IS NULL)

Detecta pacientes inactivos.

```
SELECT p.id_paciente,
       p.nombre || ' ' || p.apellido AS paciente,
       p.telefono, p.correo, p.fecha_registro
FROM paciente p
LEFT JOIN cita c ON p.id_paciente = c.id_paciente
WHERE c.id_cita IS NULL
ORDER BY p.fecha_registro;
```

Consulta 7: Historial completo (6 tablas)

INNER JOIN cruzando paciente-cita-consulta-dentista-servicio-pago.

```
SELECT p.nombre || ' ' || p.apellido AS paciente,
       ci.fecha_cita,
       d.nombre || ' ' || d.apellido AS dentista,
       s.nombre_servicio, co.diagnostico,
       pa.monto, pa.metodo_pago
FROM paciente p
INNER JOIN cita ci ON p.id_paciente = ci.id_paciente
INNER JOIN consulta co ON ci.id_cita = co.id_cita
INNER JOIN dentista d ON ci.id_dentista = d.id_dentista
INNER JOIN servicio s ON co.id_servicio = s.id_servicio
INNER JOIN pago pa ON co.id_consulta = pa.id_consulta
ORDER BY p.apellido, ci.fecha_cita;
```

Consulta 8: Reporte mensual por dentista

SUM + COUNT + COALESCE + LEFT JOIN + GROUP BY.

```
SELECT d.nombre || ' ' || d.apellido AS dentista,
       e.nombre_especialidad,
       COUNT(DISTINCT ci.id_cita) AS citas_atendidas,
       COALESCE(SUM(pa.monto), 0) AS ingresos
FROM dentista d
INNER JOIN especialidad e
       ON d.id_especialidad = e.id_especialidad
LEFT JOIN cita ci ON d.id_dentista = ci.id_dentista
       AND ci.estado = 'ATENDIDA'
LEFT JOIN consulta co ON ci.id_cita = co.id_cita
LEFT JOIN pago pa ON co.id_consulta = pa.id_consulta
GROUP BY d.id_dentista, d.nombre, d.apellido,
         e.nombre_especialidad
ORDER BY ingresos DESC;
```

6. Seguridad, roles y privilegios DCL

Se implementaron 3 roles con privilegios diferenciados aplicando el principio de mínimo privilegio.

Matriz de roles y privilegios

Tabla	Administrador	Odontólogo	Recepcionista
rol	ALL	—	—
usuario	ALL	—	—
especialidad	ALL	SELECT	SELECT
dentista	ALL	SELECT	SELECT
paciente	ALL	SELECT	SEL,INS,UPD
servicio	ALL	SELECT	SELECT
cita	ALL	SEL,UPD	SEL,INS,UPD
consulta	ALL	SEL,INS,UPD	—
pago	ALL	—	—
bitacora	ALL	—	—

Script DCL

```

CREATE ROLE claudent_admin WITH LOGIN PASSWORD 'Admin2025$Clau';
CREATE ROLE claudent_odontologo WITH LOGIN PASSWORD 'Odont2025$Clau';
CREATE ROLE claudent_recepcionista WITH LOGIN PASSWORD 'Recep2025$Clau';

GRANT USAGE ON SCHEMA claudent TO claudent_admin,
      claudent_odontologo, claudent_recepcionista;

-- Administrador: acceso total
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA claudent
      TO claudent_admin;

-- Odontologo: lectura clinica + escritura consultas
GRANT SELECT ON claudent.paciente, claudent.cita,
      claudent.consulta, claudent.servicio, claudent.especialidad,
      claudent.dentista TO claudent_odontologo;
GRANT INSERT, UPDATE ON claudent.consulta TO claudent_odontologo;
GRANT UPDATE(estado) ON claudent.cita TO claudent_odontologo;

-- Recepcionista: gestion administrativa
GRANT SELECT ON claudent.paciente, claudent.cita,
      claudent.servicio, claudent.dentista, claudent.especialidad
      TO claudent_recepcionista;
GRANT INSERT, UPDATE ON claudent.paciente TO claudent_recepcionista;
GRANT INSERT, UPDATE ON claudent.cita TO claudent_recepcionista;

-- Revocar accesos no autorizados
REVOKE ALL ON claudent.usuario
      FROM claudent_odontologo, claudent_recepcionista;
REVOKE ALL ON claudent.pago FROM claudent_recepcionista;

```

7. Pruebas de seguridad documentadas

Se ejecutaron 5 pruebas con SET ROLE para simular el acceso de cada perfil operativo.

Prueba 1: Admin consulta todas las tablas

Resultado: PERMITIDO

```
SET ROLE claudent_admin;
SELECT * FROM claudent.usuario LIMIT 3;
SELECT * FROM claudent.pago LIMIT 3;
-- Resultado: Consultas exitosas
RESET ROLE;
```

Prueba 2: Odontologo consulta pacientes

Resultado: PERMITIDO

```
SET ROLE claudent_odontologo;
SELECT * FROM claudent.paciente LIMIT 3;
-- Resultado: Datos visibles correctamente
RESET ROLE;
```

Prueba 3: Recepcionista crea cita

Resultado: PERMITIDO

```
SET ROLE claudent_recepcionista;
INSERT INTO claudent.cita(id_paciente, id_dentista,
    fecha_cita, hora_cita, motivo)
VALUES (2, 1, '2025-10-01', '09:00', 'Limpieza');
-- Resultado: INSERT 0 1
RESET ROLE;
```

Prueba 4: Recepcionista intenta ver pagos

Resultado: BLOQUEADO

```
SET ROLE claudent_recepcionista;
SELECT * FROM claudent.pago;
-- ERROR: permission denied for table pago
RESET ROLE;
```

Prueba 5: Odontologo intenta crear usuarios

Resultado: BLOQUEADO

```
SET ROLE claudent_odontologo;
INSERT INTO claudent.usuario(id_rol, nombre_usuario,
    correo, password_hash)
VALUES (2, 'Intruso', 'x@mail.com', 'hash');
-- ERROR: permission denied for table usuario
RESET ROLE;
```